HuellazGO

Guia completa de notificaciones para mascotas perdidas

Version v1.0 - enfoque MVP, anti-spam, deduplicacion, confianza y Web Push

Idea central

La notificacion no debe ser “mira otro reporte”. Debe sentirse como: “tu podrias ayudar justo aqui”. Por eso el sistema debe notificar por cercania, relevancia y confianza, no por simple existencia de eventos.

Este documento convierte la idea de notificaciones de HuellazGO en una guia accionable. La prioridad es dejar claras las reglas de producto, la logica tecnica, el manejo de miles de “Lo vi”, la deduplicacion para que una persona no reciba la misma alerta dos veces, y las recomendaciones para implementarlo sin volver castrosa la app.

Resumen ejecutivo

- Para HuellazGO v1.0, centrado solo en mascotas perdidas, la estrategia recomendada es usar notificaciones Web Push por cercania y relevancia.
- La alerta inicial de mascota perdida debe enviarse solo a usuarios dentro de un radio razonable, por defecto 3 km.
- Una persona no debe recibir dos veces la alerta inicial del mismo reporte. Esto se resuelve con historial de notificaciones y una llave de deduplicacion.
- Los avistamientos tipo “Lo vi” deben guardarse inmediatamente, pero no necesariamente notificarse al instante.
- Usuarios normales se agrupan en ventanas de 5 minutos; usuarios destacados, rescatistas o negocios verificados pueden disparar notificaciones inmediatas.
- Si hay demasiados avistamientos, el sistema cambia a modo resumen o digest para no saturar al duenio.
- La ubicacion exacta no debe mostrarse en la notificacion; solo una zona aproximada. El mapa detallado se muestra dentro del reporte.
- Todo envio debe registrarse en notification_logs para evitar spam, auditar comportamiento y resolver errores.

Regla reina

No notifiques cada evento. Notifica cuando el evento cambie la probabilidad de encontrar a la mascota.

Indice sugerido

- 1. Objetivo del sistema de notificaciones
- 2. Tipos de notificaciones de v1.0
- 3. Radio, cercania y prioridad
- 4. Privacidad y ubicacion aproximada
- 5. Deduplicacion: no avisar dos veces lo mismo
- 6. Manejo de muchos “Lo vi”
- 7. Confianza, usuarios destacados y negocios verificados
- 8. Flujo tecnico recomendado
- 9. Modelo de base de datos
- 10. Algoritmos y reglas de decision
- 11. Textos recomendados para notificaciones
- 12. Posibles problemas y soluciones
- 13. Roadmap de implementacion
- 14. Checklist final

1. Objetivo del sistema de notificaciones

El objetivo no es simplemente avisar que existe un reporte. El objetivo es aumentar la probabilidad de recuperar una mascota perdida sin saturar a los usuarios ni al duenio. Una notificacion bien enviada puede movilizar a alguien que esta cerca; una notificacion mal enviada puede hacer que el usuario desactive permisos para siempre.

Principio	Que significa en HuellazGO	Por que importa
Cercania	Notificar solo a usuarios cerca del ultimo punto relevante.	Una persona lejos rara vez puede ayudar en el momento.
Relevancia	Mandar alertas cuando hay informacion accionable.	Evita ruido y mantiene la app util.
Confianza	Dar mas peso a usuarios verificados o con historial positivo.	No todos los avistamientos tienen el mismo valor.
No duplicar	Un usuario no recibe la misma alerta inicial del mismo reporte dos veces.	Evita spam y frustracion.
Privacidad	Mostrar zona aproximada en push; detalles solo dentro del reporte.	Protege al duenio y evita exposicion innecesaria.
Escalabilidad	Agrupar miles de “Lo vi” en resumentes.	Permite crecer sin destruir la experiencia.

La filosofia debe ser: HuellazGO no grita mas fuerte, HuellazGO avisa mejor. Si la app manda diez notificaciones irrelevantes, la onceava, que tal vez era importante, ya no sera escuchada.

2. Tipos de notificaciones de v1.0

Para la primera version publica, conviene limitar el sistema a pocas notificaciones claras. Menos tipos significa menos bugs, menos confusion, menos reglas raras y mejor control del spam.

Tipo	Destinatario	Cuando se manda	Frecuencia maxima recomendada
Mascota perdida cerca	Usuarios cercanos suscritos	Cuando se publica un reporte nuevo dentro de su radio.	1 vez por usuario por reporte.
Avistamiento para el duenio	Duenio del reporte	Cuando alguien reporta “Lo vi”.	Normal: resumen cada 5 min. Verificado: inmediato.
Nuevo avistamiento cercano	Usuarios cerca del nuevo punto	Cuando el avistamiento cambia	1 cada 12-24 h por reporte, salvo evento

la zona probable. critico.

A) Mascota perdida cerca

🐾 Mascota perdida cerca de ti
“Luna” fue vista por ultima vez cerca de tu zona.
Ver reporte →

B) Avistamiento para el duenio

👀 Posible avistamiento de Luna
Alguien reporto haberla visto recientemente.
Ver ubicacion →

C) Actualizacion de zona

📍 Nuevo avistamiento cercano
Una mascota perdida fue vista recientemente cerca de ti.
Ver detalles →

Recomendacion de producto

En v1.0 no metas notificaciones de “noticias”, “voluntario destacado”, “ranking”, “nuevas funciones” o mensajes comerciales. Primero gana confianza: alertas utiles, pocas y claras.

3. Radio, cercania y prioridad

La cercania es el filtro principal. No tiene sentido avisar a todo Merida cada vez que se pierde un perro en una colonia. El radio debe cambiar segun prioridad, calidad del reporte y contexto.

Caso	Radio sugerido	Comentario
Reporte normal	2 a 3 km	Recomendado por defecto: 3 km.
Reporte urgente	5 km	Mascota vulnerable, medicamento, cachorro, adulto mayor o riesgo alto.
Radio manual maximo	10 km	Solo si el duenio lo elige o si moderacion lo aprueba.
Reporte viejo o incompleto	0 a 1 km o sin push	Puede mostrarse en mapa sin mandar push.

3.1 Clasificacion de prioridad

Prioridad	Condiciones	Accion recomendada

Alta	Se perdio hace menos de 6 horas; tiene foto; ubicacion clara; es cachorro, adulto mayor o requiere medicamento; hay avistamientos recientes.	Push a 5 km; destacar en mapa; permitir actualizaciones cercanas.
Media	Se perdio hace menos de 48 horas; tiene foto; descripcion util.	Push a 3 km; mostrar en feed local.
Baja	Reporte viejo; sin foto; ubicacion vaga; poca informacion.	Mostrar en mapa/feed, pero no mandar push o usar radio reducido.

3.2 Formula simple de prioridad

No necesitas inteligencia artificial en v1.0. Puedes usar puntos simples y transparentes.

```
priority_score = 0

if lost_hours <= 6: priority_score += 30
elif lost_hours <= 48: priority_score += 15

if has_photo: priority_score += 20
if has_clear_location: priority_score += 20
if is_vulnerable_pet: priority_score += 20
if has_recent_sightings: priority_score += 20

if priority_score >= 70: priority = 'high'
elif priority_score >= 35: priority = 'medium'
else: priority = 'low'
```

3.3 Decision de radio segun prioridad

```
if priority == 'high': radius_km = 5
elif priority == 'medium': radius_km = 3
else: radius_km = 1  # o no enviar push
```

4. Privacidad y ubicacion aproximada

En una notificacion push no deberia aparecer la ubicacion exacta. Una push puede verse en pantalla bloqueada, puede ser vista por alguien mas y no necesita mostrar datos sensibles para ser util.

No recomendado	Recomendado
Se perdio en Calle 21 #123 x 40 y 42.	Se perdio cerca de Francisco de Montejo.
Ubicacion exacta: 20.999123, -89.612345.	Ultima zona vista: Col. Centro.

Foto + direccion exacta + telefono en push.	Foto opcional + zona aproximada + boton Ver reporte.

Dentro del reporte, despues de que el usuario abre HuellazGO, si se pueden mostrar mas detalles, siempre cuidando permisos, privacidad y consentimiento del duenio.

Regla de seguridad

La notificacion debe orientar, no exponer. El detalle sensible vive dentro del reporte, no en la pantalla bloqueada.

5. Deduplicacion: no avisar dos veces lo mismo

Esta es una de las reglas mas importantes que debe quedar explicita: una persona no debe recibir dos veces la alerta inicial del mismo reporte. El sistema puede recalcular usuarios cercanos muchas veces, pero eso no significa que deba volver a notificar a los mismos usuarios.

Regla obligatoria

Usuario X + Reporte Y + Tipo de notificacion “mascota perdida cerca” = maximo una notificacion inicial.

5.1 Ejemplo practico

4:00 PM - Se publica el reporte de Luna cerca de Francisco de Montejo.
Ana vive dentro de 3 km y recibe la alerta inicial.

4:20 PM - El sistema vuelve a revisar usuarios cercanos.
Ana sigue dentro del radio, pero ya recibio esa alerta.
Resultado: NO se le vuelve a mandar.

6:00 PM - Hay un nuevo avistamiento de Luna a 200 m de Ana.
Esto ya no es la misma alerta inicial; es una actualizacion importante.
Resultado: SI se puede mandar una nueva notificacion, si cumple reglas anti-spam.

5.2 La clave tecnica: notification_logs + dedupe_key

Cada intento de notificacion debe generar una llave de deduplicacion. Si la llave ya existe para ese usuario, no se envia de nuevo.

dedupe_key para alerta inicial:
lost_pet_nearby:<report_id>

Ejemplo:
user_id = ana
report_id = luna_123
dedupe_key = lost_pet_nearby:luna_123

Si ya existe:
NO enviar otra vez.

Tipo de notificacion	Dedupe key sugerida	Regla
lost_pet_nearby	lost_pet_nearby:<report_id>	Una vez por usuario por reporte.
nearby_sighting_update	nearby_sighting_update:<report_id>:<date>	Maximo una al dia por usuario por reporte, salvo cercania critica.
sighting_summary	sighting_summary:<report_id>:<window_5_min>	Maximo una por ventana de 5 minutos al duenio.
trusted_sighting	trusted_sighting:<sighting_id>	Instantanea, pero no duplicar el mismo avistamiento.
report_found	report_found:<report_id>	Una vez.

5.3 Excepciones permitidas

- Nuevo avistamiento muy cerca del usuario.
- Avistamiento de usuario destacado, rescatista o negocio verificado.
- Cambio fuerte de zona probable.
- Reporte marcado como urgente.
- Resumen diario o recordatorio despues de 24 horas, si el usuario acepto ese tipo de alertas.

6. Manejo de muchos “Lo vi”

Si el duenio recibe miles de “Lo vi”, eso deja de ayudar. HuellazGO debe guardar todos los avistamientos, pero notificar de forma inteligente. La base es separar “guardar evento” de “enviar push”.

Separacion fundamental

Todo “Lo vi” se guarda inmediatamente. No todo “Lo vi” dispara una notificacion inmediata.

Origen del “Lo vi”	Accion de guardado	Accion de notificacion
Usuario anonimo o normal	Guardar en sightings.	Agrupar en ventana de 5 minutos.
Usuario registrado con historial positivo	Guardar en sightings.	Agrupar o notificar rapido si el trust_score es alto.
Negocio verificado	Guardar en sightings.	Notificacion inmediata al duenio.
Rescatista/destacado	Guardar en sightings.	Notificacion inmediata al duenio.

```
Usuario normal reporta “Lo vi”
   ↓
Guardar avistamiento en sightings
   ↓
No mandar push inmediatamente
   ↓
Agregar a grupo pendiente del reporte
   ↓
Cada 5 minutos:
   - contar avistamientos
   - detectar zona principal
   - mandar resumen al duenio
```

6.1 Ventanas de acumulacion

Una ventana de 5 minutos es buen punto de partida. Es lo suficientemente rapida para ayudar, pero reduce muchisimo el spam si llegan muchos reportes.

Situacion	Regla recomendada
1 avistamiento normal	Esperar hasta el cierre de la ventana de 5 minutos y mandar resumen si sigue siendo relevante.
3+ avistamientos en misma zona	Mandar resumen con zona principal.
10+ avistamientos en 10 minutos	Activar modo alta actividad.
Avistamiento verificado	Rompe la ventana y se notifica al instante.

7. Confianza, usuarios destacados y negocios verificados

No todos los avistamientos deben pesar igual. Un “Lo vi” anonimo puede ser util, pero uno de un negocio verificado, veterinaria, rescatista o usuario con historial positivo merece mas prioridad.

Tipo de usuario	trust_score sugerido	Comportamiento
Anonimo	20	Guardar y agrupar. No notificar instantaneo.
Registrado normal	40	Guardar y agrupar.
Usuario con historial positivo	60	Agrupar o subir prioridad si hay coincidencias.
Negocio verificado	80	Notificacion inmediata al duenio.
Rescatista/destacado	90	Notificacion inmediata al duenio.
Duenio del reporte	100	No aplica como fuente normal

de avistamiento.

```
if sighting.trust_score >= 80:
    send_immediate_notification_to_owner(sighting)
else:
    add_to_pending_sighting_digest(sighting, window_minutes=5)
```

Cuidado

La confianza no debe convertirse en privilegio opaco. Explica en la app que negocios/rescatistas verificados generan alertas prioritarias porque suelen tener mayor confiabilidad operativa.

8. Flujo tecnico recomendado

El flujo recomendado separa cuatro capas: evento, decision, envio y registro. Esto permite auditar, reintentar y evitar duplicados.

```
Usuario publica mascota perdida
    ↓
Backend calcula prioridad y radio
    ↓
Busca usuarios suscritos dentro del radio
    ↓
Filtra usuarios que ya recibieron la alerta inicial
    ↓
Envia Web Push
    ↓
Registra cada envio en notification_logs

Usuario reporta “Lo vi”
    ↓
Guardar avistamiento en sightings
    ↓
Si trust_score >= 80: notificar al duenio al instante
Si trust_score < 80: agrupar en digest de 5 minutos
    ↓
Opcional: si cambia la zona probable, notificar usuarios cercanos a nueva zona
```

Capa	Responsabilidad	Ejemplo
Evento	Guardar lo que paso.	Se creo reporte; se agrego avistamiento.
Decision	Evaluar si vale la pena notificar.	Esta cerca? Ya fue notificado? Es confiable?
Envio	Mandar Web Push.	Service Worker recibe payload.
Registro	Guardar resultado.	notification_logs con status

sent/failed/skipped.

9. Modelo de base de datos recomendado

El modelo puede iniciar simple, pero debe incluir desde el principio los campos necesarios para deduplicacion, preferencias, confianza y auditoria.

```
-- Suscripciones push del navegador
create table public.push_subscriptions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id) on delete cascade,
  endpoint text not null,
  p256dh text not null,
  auth text not null,
  user_agent text,
  is_active boolean not null default true,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (user_id, endpoint)
);


-- Preferencias simples de notificacion
create table public.notification_preferences (
  user_id uuid primary key references auth.users(id) on delete cascade,
  notify_lost_pets boolean not null default true,
  notify_sighting_updates boolean not null default true,
  radius_km numeric not null default 3,
  pet_type_filter text not null default 'all', -- all, dogs, cats
  quiet_hours_start time,
  quiet_hours_end time,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);


-- Registro de notificaciones: clave para anti-spam y auditoria
create table public.notification_logs (
  id uuid primary key default gen_random_uuid(),
  user_id uuid references auth.users(id) on delete cascade,
  report_id uuid not null references public.lost_pet_reports(id) on delete cascade,
  sighting_id uuid,
  notification_type text not null,
  dedupe_key text not null,
  title text not null,
  body text not null,
  status text not null default 'pending', -- pending, sent, failed, skipped
  failure_reason text,
  sent_at timestamptz,
  created_at timestamptz not null default now(),
  unique (user_id, dedupe_key)
```

```
);


-- Avistamientos
create table public.sightings (
  id uuid primary key default gen_random_uuid(),
  report_id uuid not null references public.lost_pet_reports(id) on delete cascade,
  created_by uuid references auth.users(id) on delete set null,
  seen_at timestamptz not null default now(),
  lat double precision not null,
  lng double precision not null,
  approximate_zone text,
  comment text,
  photo_url text,
  trust_score integer not null default 40,
  source_type text not null default 'normal', -- anonymous, normal, verified_business, rescuer
  status text not null default 'visible',
  created_at timestamptz not null default now()
);


-- Cola o tabla de digest para agrupar Lo vi normales
create table public.sighting_digest_queue (
  id uuid primary key default gen_random_uuid(),
  report_id uuid not null references public.lost_pet_reports(id) on delete cascade,
  window_start timestamptz not null,
  window_end timestamptz not null,
  status text not null default 'pending', -- pending, processed, skipped
  created_at timestamptz not null default now(),
  unique (report_id, window_start)
);
```

10. Algoritmos y reglas de decision

A continuacion se proponen reglas en pseudocodigo para que la implementacion sea directa.

10.1 Alerta inicial de mascota perdida

```
function notifyLostPetNearby(report):
    priority = calculatePriority(report)
    radiusKm = getRadiusByPriority(priority)

    if radiusKm <= 0:
        return

    users = findSubscribedUsersNear(report.lat, report.lng, radiusKm)

    for user in users:
        if !user.preferences.notify_lost_pets:
            continue

        dedupeKey = 'lost_pet_nearby:' + report.id
```

```
  if notificationLogExists(user.id, dedupeKey):
    continue

  sendPush(user, {
    title: '🐾 Mascota perdida cerca de ti',
    body: report.pet_name + ' fue vista por ultima vez cerca de tu zona.',
    url: '/reportes/' + report.id
  })

  insertNotificationLog(user.id, report.id, 'lost_pet_nearby', dedupeKey)
```

10.2 Avistamiento “Lo vi”

```
function handleSightingCreated(sighting):
  saveSighting(sighting)

  if sighting.trust_score >= 80:
    notifyOwnerImmediately(sighting)
  else:
    addToFiveMinuteDigest(sighting)

  if sightingIsRelevantForNearbyUsers(sighting):
    maybeNotifyNearbyUsersAboutNewZone(sighting)
```

10.3 Digest de avistamientos cada 5 minutos

```
function processSightingDigest(reportId, windowStart, windowEnd):
  sightings = getNormalSightings(reportId, windowStart, windowEnd)

  if sightings.count == 0:
    markDigestSkipped()
    return

  owner = getReportOwner(reportId)
  mainZone = calculateMainZone(sightings)

  dedupeKey = 'sighting_summary:' + reportId + ':' + windowStart

  if notificationLogExists(owner.id, dedupeKey):
    return

  if sightings.count == 1:
    title = '👀 Nuevo posible avistamiento'
    body = 'Alguien reporto haber visto a tu mascota recientemente.'
  else:
    title = '👀 ' + sightings.count + ' posibles avistamientos'
    body = 'La mayoria la ubican cerca de ' + mainZone + '.'

  sendPush(owner, { title, body, url: '/reportes/' + reportId + '/mapa' })
  insertNotificationLog(owner.id, reportId, 'sighting_summary', dedupeKey)
```

10.4 Alta actividad

```
function checkHighActivity(reportId):
  count = countSightingsInLastMinutes(reportId, 10)

  if count >= 10:
    enableHighActivityMode(reportId)
    sendOrUpdateHighActivityDigest(reportId)
```

11. Textos recomendados para notificaciones

Los textos deben ser breves, accionables y cuidadosos con privacidad. Evita meter demasiada informacion en el push.

Caso	Titulo	Cuerpo
Alerta inicial	🐾 Mascota perdida cerca de ti	“Luna” fue vista por ultima vez cerca de tu zona.
Avistamiento normal acumulado	👀 Nuevo posible avistamiento	Alguien reporto haber visto a Luna recientemente.
Varios avistamientos	👀 6 posibles avistamientos de Luna	La mayoria la ubican cerca de Francisco de Montejo.
Avistamiento confiable	🚨 Avistamiento confiable de Luna	Un usuario destacado reporto haberla visto hace unos momentos.
Negocio verificado	🏥 Avistamiento desde negocio verificado	Un negocio aliado reporto haber visto a Luna.
Alta actividad	🔥 Alta actividad en el reporte de Luna	23 avistamientos recientes la ubican cerca de Col. Mexico Norte.

Tono recomendado

Urgente, pero no alarmista. Claro, pero sin exponer datos sensibles. Util, pero no invasivo.

12. Posibles problemas y soluciones propuestas

Problema	Riesgo	Solucion propuesta
Usuarios desactivan notificaciones	Pierdes el canal principal de alerta.	Pedir permiso en el momento correcto y limitar frecuencia.
Una persona recibe la misma	La app se percibe como spam.	notification_logs + dedupe_key

alerta dos veces		unica por usuario y tipo.
Miles de “Lo vi” saturan al duenio	Ansiedad, ruido y mala experiencia.	Ventanas de 5 minutos, digest y modo alta actividad.
Reportes falsos o bromas	Desinformacion y perdida de confianza.	trust_score, reportes, moderacion y verificacion de usuarios destacados.
Ubicacion sensible expuesta	Riesgo de privacidad.	Mostrar zona aproximada en push; mapa detallado dentro de reporte.
Notificaciones fallidas	Usuarios no reciben alertas.	Registrar status failed, reintentar con limites y desactivar endpoints vencidos.
Demasiados usuarios cerca	Costo/tiempo de envio alto.	Procesar en batches y limitar por prioridad/radio.
Avistamientos dispersos	El duenio no sabe a cual creer.	Agrupar por zona y mostrar zona principal/probable.
Usuarios normales quieren prioridad	Conflicto de percepcion.	Explicar que la prioridad depende de confianza e historial, no de favoritismo.
Spam intencional de “Lo vi”	Manipulacion del mapa.	Rate limit por usuario/IP/reporte y deteccion de patrones raros.

13. Roadmap de implementacion

Para no atascarte, implementalo por fases. Primero que funcione; luego que sea inteligente; luego que sea bonito.

Fase	Objetivo	Entregables
Fase 1 - Base	Permisos y suscripcion push.	Service Worker, tabla push_subscriptions, boton activar alertas.
Fase 2 - Alerta inicial	Notificar reportes cercanos sin duplicar.	Busqueda por radio, notification_logs, dedupe_key lost_pet_nearby.
Fase 3 - Avistamientos	Guardar “Lo vi” y avisar al duenio.	Tabla sightings, notificacion al duenio.
Fase 4 - Anti-spam	Agrupar avistamientos normales.	Digest cada 5 minutos, limites de frecuencia.

Fase 5 - Confianza	Priorizar usuarios destacados/negocios.	trust_score, source_type, notificacion inmediata.
Fase 6 - Alta actividad	Resumir muchos eventos.	Modo digest extendido, zona principal, mapa de calor opcional.

MVP recomendado

Para lanzar v1.0 no necesitas mapa de calor avanzado ni IA. Necesitas: alerta inicial cercana, deduplicacion, “Lo vi” guardado, resumen al duenio y excepcion para verificados.

14. Checklist final para HuellazGO v1.0

- La app pide permiso de notificaciones solo despues de una accion relevante.
- Cada usuario tiene preferencias basicas: recibir alertas si/no y radio.
- La alerta inicial se manda solo a usuarios cercanos.
- La alerta inicial no se repite para el mismo usuario y reporte.
- La ubicacion exacta no aparece en la notificacion push.
- Todos los envios quedan registrados en notification_logs.
- Los “Lo vi” se guardan inmediatamente.
- Usuarios normales se agrupan en digest de 5 minutos.
- Usuarios destacados/negocios verificados notifican al duenio al instante.
- Hay limites de frecuencia por reporte, usuario y tipo de notificacion.
- Existe modo alta actividad si hay muchos avistamientos.
- El reporte muestra historial completo, pero la push solo resume.
- Hay manejo de endpoints fallidos o suscripciones vencidas.
- Hay rate limit para evitar abuso.

15. Consideraciones tecnicas para Web Push

Las Web Push Notifications normalmente requieren un Service Worker, llaves VAPID, una suscripcion generada por el navegador y un backend que envie el mensaje. Como HuellazGO ya puede usar Supabase, una arquitectura razonable seria guardar las suscripciones en Postgres y enviar desde una Edge Function o backend propio.

Componente	Funcion
Service Worker	Recibe la notificacion aunque la pagina no este abierta.
Push Subscription	Contiene endpoint, p256dh y auth. Se guarda en la base de datos.
Backend / Edge Function	Decide a quien mandar y ejecuta el envio.
notification_logs	Evita duplicados y permite auditar.

Preferencias	Controlan radio, tipos y permisos del usuario.

Frontend:
1. Registrar Service Worker.
2. Pedir permiso al usuario en momento oportuno.
3. Crear push subscription.
4. Guardar subscription en Supabase.

Backend:
1. Recibir evento: reporte creado o avistamiento creado.
2. Evaluar reglas de prioridad, cercania y dedupe.
3. Enviar push.
4. Registrar resultado.

16. Casos especiales que conviene definir desde el principio

Caso	Decision recomendada
El duenio edita la ubicacion del reporte	No reenviar alerta inicial a los mismos usuarios; solo notificar nuevos usuarios si la zona cambio mucho.
El reporte se marca como encontrado	Detener notificaciones relacionadas y opcionalmente avisar a quienes interactuaron.
El usuario desactiva alertas	No enviar aunque este cerca. Respetar preferencias siempre.
El usuario esta dentro de radio pero ya recibio alerta	No reenviar alerta inicial.
Nuevo avistamiento a menos de 300 m del usuario	Puede mandar actualizacion aunque ya recibio alerta inicial, sujeto a limite diario.
Usuario hace spam de “Lo vi”	Rate limit: maximo N avistamientos por reporte por hora.
Varios avistamientos desde el mismo punto	Agrupar por zona y mostrar conteo; evitar multiples push.
Endpoint push falla varias veces	Marcar suscripcion como inactiva.

17. Escenarios completos de ejemplo

Escenario 1: reporte nuevo normal

1. Duenio publica a Luna perdida en Col. Centro.
2. HuellazGO calcula prioridad media y radio de 3 km.
3. Encuentra 800 usuarios suscritos dentro del radio.

4. Filtra preferencias y dedupe.
5. Envia alerta a 620 usuarios.
6. Registra 620 filas en notification_logs.
7. Si el proceso se repite, no vuelve a notificar a esos mismos usuarios.

Escenario 2: muchos “Lo vi” normales

1. En 5 minutos llegan 12 avistamientos normales.
2. Todos se guardan en sightings.
3. No se mandan 12 push al duenio.
4. Al cerrar la ventana, se calcula zona principal.
5. El duenio recibe: “12 posibles avistamientos de Luna cerca de Col. Centro”.
6. Dentro del reporte puede ver la lista completa.

Escenario 3: negocio verificado reporta avistamiento

1. Veterinaria verificada reporta haber visto a Luna.
2. trust_score = 80.
3. HuellazGO notifica inmediatamente al duenio.
4. El evento tambien queda en sightings.
5. Si despues llegan usuarios normales, esos se agrupan en digest.

Escenario 4: avistamiento cercano a un usuario ya notificado

1. Ana recibio alerta inicial de Luna a las 4:00 PM.
2. A las 6:00 PM aparece un avistamiento a 200 m de Ana.
3. Aunque Ana ya recibio la alerta inicial, este es un evento nuevo y cercano.
4. HuellazGO puede mandarle una actualizacion, si no ha superado su limite diario.

Conclusion

Para HuellazGO, el exito de las notificaciones no depende de mandar mas, sino de mandar mejor. La estrategia recomendada para v1.0 es clara: radio de 3 km por defecto, alerta inicial una sola vez por usuario y reporte, ubicacion aproximada en push, “Lo vi” guardado siempre pero agrupado cada 5 minutos, y prioridad inmediata para usuarios confiables o negocios verificados.

Con estas reglas, HuellazGO se mantiene util, seguro y escalable. El usuario siente que la app le avisa cuando realmente puede ayudar, y el duenio recibe informacion accionable sin ahogarse en ruido.

Frase guia

HuellazGO no debe competir por atencion. Debe ganar confianza.